

ESPECIFICAÇÃO NORMATIVA

Especificação de Segurança, Governança e Auditabilidade

API Management Tax — Fundação de Confiança e Portões de Produção

Documento: AMT-GOV-SEC-001

Versão: 0.1

Data: 14 de agosto de 2026

Classificação: CONFIDENCIAL

Status: Minuta normativa para validação

Aplicação: GitHub, VS Code, WSL2/Linux, Docker, PostgreSQL, Prisma ORM/Migrate e Prisma Studio

DECISÃO DE CONTROLE Nenhum dado tributário real, credencial de produção ou operação produtiva será admitido antes da aprovação dos portões definidos neste documento. Segurança, isolamento, auditoria e reprodutibilidade são requisitos de produto, não atividades posteriores.

Público-alvo: responsável pelo produto, Management Tax, Engenharia, Segurança da Informação, Privacidade/DPO, Jurídico, Operações, Auditoria e aprovadores de produção.

Relação documental: este documento complementa a Especificação Consolidada do API Management Tax e prevalece para requisitos de proteção, governança, auditoria, migrações e regras de negócio.

0. Controle do documento

Campo	Definição
Proprietário	Responsável pelo produto API Management Tax
Aprovadores mínimos	Produto, Engenharia, Segurança, Management Tax e Privacidade/Jurídico quando aplicável
Periodicidade de revisão	Trimestral e também após incidente relevante, alteração arquitetural material ou mudança regulatória aplicável
Regra de mudança	Toda alteração deve ocorrer por pull request, com histórico, justificativa, revisão e aprovação dos proprietários definidos em CODEOWNERS
Evidência oficial	Versão aprovada no repositório privado, associada a commit/tag e hash do artefato publicado

0.1 Convenções normativas

- DEVE / NÃO DEVE: requisito obrigatório para aprovação do portão correspondente.
- DEVERIA: controle recomendado; eventual exceção exige risco documentado, responsável e prazo de correção.
- PODE: opção permitida, desde que não reduza controles obrigatórios.
- Evidência: artefato verificável que demonstra implementação e eficácia do controle.
- Exceção: desvio temporário formalmente aprovado; nunca substitui correção definitiva.

0.2 Hierarquia de autoridade

1. Lei, regulação, obrigação contratual e política corporativa aplicável.
2. Esta especificação e decisões arquiteturais aprovadas (ADRs).
3. Regras de negócio publicadas e versionadas.
4. Procedimentos operacionais, runbooks e instruções de trabalho.

REGRA DE CONFLITO Quando houver conflito, aplica-se o requisito mais restritivo até decisão formal dos responsáveis por Segurança, Jurídico/Privacidade e Produto.

1. Decisão executiva e política inegociável

O API Management Tax será construído como sistema auditável de suporte à gestão tributária. A plataforma não substitui validação profissional e não pode converter fonte preliminar em posição tributária aprovada sem fluxo formal de revisão.

- Acesso é negado por padrão e concedido somente pelo menor privilégio necessário.
- Todo registro sujeito a segregação deve possuir contexto explícito de tenant e jurisdição.
- Toda decisão material deve ser reproduzível com a versão exata da regra e das fontes utilizadas.
- Todo evento sensível deve produzir trilha de auditoria independente da vontade do usuário comum.
- Migrações de banco e alterações de regras devem ser versionadas, revisadas, testadas e aprovadas.
- Prisma Studio é ferramenta de desenvolvimento local; não é mecanismo operacional de produção.

- McAfee reduz riscos no host Windows, mas não substitui controles de WSL, containers, dependências, identidade, aplicação ou banco de dados.

1.1 Condição para iniciar desenvolvimento funcional

Endpoints, telas e integrações funcionais podem ser prototipados somente com dados sintéticos após o Gate 1. Dados reais permanecem proibidos até a aprovação do Gate 2. Produção permanece proibida até o Gate 7.

2. Escopo, premissas e fronteiras de confiança

2.1 Escopo obrigatório

- Código-fonte, infraestrutura como código, pipelines, imagens de container e dependências.
- Identidades humanas, contas técnicas, tokens, chaves, certificados e segredos.
- Banco PostgreSQL, Prisma ORM/Migrate, Prisma Studio, backups, exports e réplicas.
- Regras tributárias, fontes legais, pareceres, evidências, anexos e decisões.
- Ambientes de desenvolvimento, integração, homologação, produção e recuperação.
- GitHub, VS Code conectado ao WSL2, Docker e estações autorizadas.

2.2 Fronteiras de confiança

Fronteira	Risco principal	Controle mínimo
Usuário → API	Uso indevido de identidade ou escopo	SSO, MFA, sessão curta, autorização contextual, rate limit e auditoria
API → banco	Bypass de isolamento ou privilégio excessivo	Conta runtime restrita, RLS, parâmetros, TLS e credencial rotacionável
CI/CD → ambiente	Comprometimento da cadeia de entrega	OIDC, aprovação de ambiente, artefato assinado, SHA fixo e permissões mínimas
Windows → WSL2	Exposição por host ou montagem	Disco protegido, projeto no filesystem Linux, extensão confiável e portas locais
Container → host	Escalada e movimento lateral	Usuário não root, capabilities removidas, filesystem somente leitura e sem Docker socket
Sistema → exportação	Exfiltração de dados	Autorização específica, marca d'água/classificação, expiração e auditoria
Regra → decisão	Resultado não reproduzível	Versão imutável, fonte vinculada, checksum, vigência e registro da execução

3. Classificação e tratamento de dados

Classe	Exemplos	Tratamento mínimo
Público	Legislação oficialmente pública já validada	Integridade, fonte, data de captura e licença/termos de uso

Classe	Exemplos	Tratamento mínimo
Interno	Roadmaps, arquitetura não sensível e backlog	Acesso autenticado e compartilhamento controlado
Confidencial	Posições tributárias, pareceres, contratos, dados de entidades	Criptografia, menor privilégio, log de acesso e proibição em repositório público
Restrito	Credenciais, chaves, dados pessoais sensíveis, investigações	Vault, acesso just-in-time, dupla aprovação quando aplicável, retenção mínima e alerta

- Todo objeto persistido deve possuir classificação de informação explícita ou herdada de política verificável.
- Logs, mensagens de erro, telemetria e dados de teste não devem conter segredos, tokens, senhas ou conteúdo integral de pareceres.
- Dados reais não devem ser copiados para desenvolvimento. Quando necessário, utilizar dados sintetizados ou anonimizados com validação de irreversibilidade.
- Exportações devem possuir propósito, destinatário, prazo de validade, classificação e evento de auditoria.

4. Identidade, autenticação e autorização

4.1 Requisitos de identidade

- Contas humanas devem usar identidade corporativa, MFA resistente a phishing quando disponível e proibição de contas compartilhadas.
- Contas técnicas devem ser não interativas, possuir proprietário, finalidade, escopo, validade e rotação definidos.
- Acesso privilegiado deve ser temporário, justificado e revisado; elevação e uso devem ser auditados.
- Desligamento ou mudança de função deve revogar acessos dentro do SLA corporativo e invalidar sessões ativas quando o risco exigir.

4.2 Modelo de autorização

A autorização deve combinar RBAC para funções estáveis e ABAC para contexto: tenant, jurisdição, entidade legal, escritório, classificação, materialidade, status do fluxo e relação com o caso.

Papel	Permissões típicas	Restrições
Leitor tributário	Consultar conteúdo aprovado e casos autorizados	Sem edição, aprovação, exportação ampla ou administração
Analista	Criar minutas, anexar fontes e executar simulações	Não publica regra nem aprova a própria posição
Revisor tributário	Revisar conteúdo e registrar validação profissional	Não altera evidência original; conflito de interesse controlado
Aprovador	Aprovar posição/regra dentro de alçada	Dupla aprovação em mudanças materiais ou retroativas

Papel	Permissões típicas	Restrições
Administrador do tenant	Gerenciar usuários e parâmetros do próprio tenant	Sem acesso automático a conteúdo restrito; atos auditados
Operações/SRE	Operar infraestrutura e responder incidentes	Sem permissão funcional implícita para conteúdo tributário
Auditor	Consulta somente leitura de trilhas e evidências	Sem alteração de registros ou regras

4.3 Testes obrigatórios de autorização

- Usuário autorizado conclui a operação e gera evidência.
- Usuário sem papel adequado recebe negação sem revelar existência, conteúdo ou identificadores sensíveis.
- Usuário do tenant A não acessa, pesquisa, exporta ou infere dados do tenant B.
- Administrador não herda acesso a conteúdo restrito sem política explícita.
- Mudança de papel invalida autorização cacheada e é refletida dentro do SLA definido.
- Ações em lote, jobs assíncronos e integrações aplicam os mesmos filtros e políticas.

5. Isolamento de dados e PostgreSQL RLS

Todo registro sujeito a segregação deve conter `tenant_id` não nulo e, conforme o domínio, `organization_id`, `jurisdiction_id`, `legal_entity_id`, `office_id` e `information_classification`. O contexto deve ser propagado do token/autorização até a transação de banco, sem aceitar `tenant_id` fornecido pelo cliente como fonte de autoridade.

- RLS deve estar habilitado e forçado nas tabelas multitenant; a conta runtime não pode possuir `BYPASSRLS` nem propriedade irrestrita das tabelas.
- A conta de migração deve ser separada da conta de execução da API e não deve ser usada por serviços em tempo de execução.
- Consultas administrativas cross-tenant exigem função específica, justificativa, escopo limitado e evento de auditoria crítico.
- Cache, busca, fila, object storage, relatórios, anexos e backups devem preservar a mesma fronteira de isolamento.
- Porta do PostgreSQL não deve ser publicada na internet; em desenvolvimento, exposição somente em localhost e com credenciais não produtivas.

5.1 Critérios de isolamento

Cenário	Resultado esperado	Evidência
Consulta direta com tenant incorreto	Zero linhas e nenhum metadado revelado	Teste automatizado de integração
Tentativa de bypass por parâmetro	Contexto do token prevalece; requisição rejeitada	Teste negativo de API
Job assíncrono	Contexto do tenant é obrigatório e validado	Teste de fila + auditoria correlacionada

Cenário	Resultado esperado	Evidência
Exportação	Somente escopo autorizado, arquivo classificado e expiração	Teste funcional e evento de export
Backup/restauração	Restauração preserva segregação e acesso	Relatório de restore em ambiente isolado
Administrador	Sem leitura implícita de conteúdo	Matriz de autorização e teste adversarial

6. Auditoria imutável e evidência de integridade

A auditoria deve ser append-only para usuários e serviços comuns. Correções não apagam eventos: produzem novo evento relacionado. O armazenamento deve possuir retenção, controle de acesso, sincronização temporal e mecanismo de detecção de adulteração.

6.1 Evento mínimo

Campo	Obrigação
event_id / event_type	Identificador global e tipo estável/versionado
occurred_at_utc	Data/hora UTC com precisão e origem temporal conhecida
actor	Usuário/serviço, papel, tenant e método de autenticação
scope	Tenant, jurisdição, entidade legal, recurso e classificação
action / result	Ação solicitada, permitido/negado, status e motivo controlado
before / after	Diferenças relevantes ou referências seguras; nunca segredos
reason / source	Justificativa, origem da mudança e ticket/PR quando aplicável
correlation_id	Correlação entre gateway, aplicação, banco, job e pipeline
rule_version	Versão exata da regra usada na decisão, quando aplicável
integrity	Checksum/hash e referência ao encadeamento/lote de integridade

- Leituras de conteúdo restrito, exportações, alterações de permissão, publicação de regra, migração, restauração e ações administrativas devem ser eventos sensíveis.
- Eventos sensíveis devem gerar monitoramento ou alerta proporcional ao risco.
- Logs operacionais não substituem trilha de auditoria; identidade, banco, infraestrutura e CI/CD também devem emitir evidências correlacionáveis.
- Retenção e legal hold devem impedir descarte enquanto houver obrigação legal, investigação ou auditoria.

7. Baseline de plataforma: GitHub, VS Code, WSL2 e Docker

7.1 GitHub

- Repositório privado, MFA/passkey, membros mínimos, revisão periódica de colaboradores e proibição de credenciais em código ou issues.
- Branch principal protegida por ruleset: pull request, revisores obrigatórios, CODEOWNERS, checks aprovados, conversa resolvida e bloqueio de force-push/deletion.
- Secret scanning, dependency review, atualização automatizada e análise estática habilitadas conforme plano/licença disponível.
- Commits e tags de release devem ser assinados quando tecnicamente viável; releases devem referenciar artefato e SBOM.
- GitHub Actions deve usar permissões mínimas, actions fixadas por SHA completo, OIDC para nuvem e aprovação de environment para produção.

7.2 VS Code e WSL2

- O projeto deve residir no filesystem Linux do WSL2, não em /mnt/c, reduzindo diferenças de permissão, desempenho e observação de arquivos.
- VS Code deve conectar via Remote WSL, usar Workspace Trust e permitir somente extensões aprovadas; extensão nova exige análise de origem e permissões.
- O arquivo .env não deve ser versionado; exemplos devem usar valores fictícios e nomes sem segredos.
- Serviços de desenvolvimento devem escutar em localhost; publicação de porta deve ser explícita e temporária.
- Distribuição WSL, Docker, VS Code, extensões e pacotes devem permanecer atualizados segundo SLA de vulnerabilidade.

7.3 Docker

- Containers devem executar como usuário não root, com capabilities removidas, filesystem somente leitura quando compatível e diretórios temporários explícitos.
- O Docker socket não deve ser montado na aplicação; acesso privilegiado e host network são proibidos salvo exceção aprovada.
- Imagens devem usar base mínima, versão/digest fixado, build multi-stage, usuário explícito, healthcheck e ausência de segredos em layers.
- Tags latest são proibidas em produção. Imagens devem ser escaneadas, gerar SBOM e, no fluxo produtivo, ser assinadas e verificadas.
- Bancos e serviços internos devem usar rede interna; somente o gateway necessário expõe porta e aplica TLS/rate limiting.

8. Segredos e criptografia

- Segredos produtivos devem permanecer em cofre gerenciado, nunca em Git, imagem, compose, log, documentação, prompt, issue ou histórico de terminal compartilhado.
- Credenciais devem ter menor escopo, validade curta quando possível, rotação e revogação testada.
- Dados em trânsito devem usar TLS; dados confidenciais e backups devem ser criptografados em repouso com gestão de chaves separada do armazenamento.
- Chaves devem possuir proprietário, finalidade, versão, rotação, revogação e registro de uso. A aplicação não deve registrar material criptográfico.
- Detecção de segredo em commit e pipeline deve bloquear a entrega. Segredo exposto deve ser revogado, não apenas removido do arquivo.

9. Prisma, Prisma Studio e governança do banco

POLÍTICA DO PRISMA STUDIO Uso permitido somente em desenvolvimento local, com dados fictícios e credenciais sem acesso a produção. Não é ferramenta de correção operacional, alteração de schema, aprovação ou administração produtiva.

- Alterações de schema devem ocorrer por Prisma Migrate e arquivos versionados; alterações manuais no banco são proibidas fora de procedimento emergencial aprovado.
- Produção deve usar prisma migrate deploy por pipeline controlado; prisma migrate dev é restrito ao ambiente local.
- Correções de dados produtivos devem ocorrer pela aplicação auditada ou script versionado, revisado, testado, aprovado e associado a ticket.
- A conta usada pelo Prisma Studio local deve apontar somente para banco local/sintético e possuir nome visualmente distinto de qualquer conta produtiva.
- A conexão produtiva deve ser tecnicamente inacessível a partir do Studio por ausência de credencial, rede e permissão.

10. Governança de migrações

10.1 Fluxo obrigatório

1. Propor alteração de modelo e impacto em dados, APIs, regras, RLS, índices, locks e rollback/forward-fix.
2. Gerar migração no ambiente local e revisar SQL produzido; nenhuma migração aplicada pode ser reescrita.
3. Abrir pull request com migration plan, classificação de risco, evidências e responsáveis.
4. Executar migração em banco descartável desde zero e também sobre cópia sintética do schema anterior.
5. Validar compatibilidade entre versão antiga/nova da aplicação e expand-contract quando houver rollout gradual.
6. Executar em homologação, medir duração/locks, testar backup e plano de recuperação.
7. Obter aprovação exigida pelo risco e executar prisma migrate deploy via pipeline.

8. Verificar saúde, integridade, RLS, auditoria e métricas; registrar resultado e eventual forward-fix.

10.2 Política expand-contract

Fase	Ação	Proibição
Expandir	Adicionar estrutura compatível, nullable/default seguro, índice concorrente quando aplicável	Remover ou renomear campo ainda usado
Migrar	Backfill em lotes, observabilidade, idempotência e limite de carga	Transação longa sem estimativa/controle
Trocar	Aplicação passa a ler/escrever novo modelo com feature flag se necessário	Corte sem compatibilidade validada
Contrair	Remover legado somente após prova de não uso e janela aprovada	Drop destrutivo no mesmo release da expansão

- Checksums de migrações aplicadas devem ser verificados. Divergência bloqueia deploy e exige investigação.
- Migração destrutiva, retroativa ou de alto volume exige aprovação adicional de Banco/Plataforma, Segurança e proprietário do dado.
- Rollback físico pode ser inseguro; o plano deve priorizar forward-fix, compatibilidade e restauração somente quando os critérios forem satisfeitos.

11. Regras de negócio versionadas

Toda regra tributária que influencie cálculo, classificação, prazo, obrigação, alçada, alerta ou decisão deve ser tratada como artefato versionado, explicável e reproduzível.

11.1 Modelo mínimo

Campo	Finalidade
rule_id / version	Identidade estável e versão imutável
tenant_id opcional	Regra global ou escopo específico autorizado
jurisdiction_id / rule_type	País/região e categoria funcional
status	Draft, Review, Validated, Approved, Published, Superseded ou Archived
effective_from / effective_to	Vigência legal separada da data de publicação
legal_basis / source_id	Base legal, fonte oficial, data de captura e confiabilidade
parameters / expression	Parâmetros e expressão determinística ou implementação referenciada
approval	Status, aprovadores, datas, alçada e segregação
supersedes_version	Relação explícita com versão anterior
checksum	Prova de integridade do conteúdo publicado

11.2 Ciclo de vida

Draft → Review → Validated (especialista) → Approved → Published → Superseded → Archived.

- Versão Published é imutável. Qualquer alteração cria nova versão e preserva a anterior.
- Aprovação e publicação são eventos distintos; vigência legal pode anteceder ou suceder publicação e deve ser explícita.
- Retroatividade exige justificativa, avaliação de impacto, dupla aprovação e reprocessamento controlado.
- Conteúdo preliminar, fonte não confirmada ou resultado de pesquisa não pode ser publicado como regra aprovada.
- Toda decisão deve armazenar rule_id, version, checksum, inputs relevantes, data de referência e resultado.
- O sistema deve reproduzir decisão histórica sem substituir a regra usada pela versão corrente.

11.3 Critérios de publicação

Critério	Evidência obrigatória
Fonte	Fonte oficial ou classificação de confiabilidade, captura e referência
Validação	Parecer/revisão do especialista identificado
Teste	Casos normais, limites, exceções, datas e regressão histórica
Alçada	Aprovadores compatíveis com materialidade e jurisdição
Explicabilidade	Descrição legível, fórmula/parâmetros e razão da mudança
Integridade	Checksum e vínculo com commit/release do motor quando aplicável

12. SDLC, CI/CD e cadeia de suprimentos

- Todo código entra por pull request com revisão independente; autor não aprova sozinho mudança material de segurança, migração ou regra.
- Pipeline deve executar lint, testes unitários, integração, isolamento, migração, SAST, SCA, secret scan, container scan e geração de SBOM.
- Dependências devem ser fixadas por lockfile, ter origem confiável e passar por política de vulnerabilidade/licença.
- Build deve ser reproduzível e gerar artefato imutável promovido entre ambientes; produção não recompila código diferente.
- Segredos não devem ser disponibilizados a pull requests não confiáveis. Permissões do token de automação devem ser read-only por padrão.
- Release deve possuir versão, commit, imagem/digest, SBOM, migrações, regras compatíveis, aprovações e notas de mudança.

13. Backup, recuperação, retenção e descarte

- Backups devem ser criptografados, segregados do ambiente de origem, monitorados e protegidos contra alteração/exclusão indevida.
- RPO e RTO devem ser definidos por ambiente e aprovados pelo negócio; restauração deve ser testada periodicamente.
- Teste de restore deve validar schema, migrações, RLS, anexos, regras, auditoria, chaves e consistência funcional.
- Retenção deve considerar obrigação legal, fiscal, contratual, privacidade e legal hold; descarte deve ser verificável e auditado.
- Backups não devem ser usados como atalho para disponibilizar dados reais em desenvolvimento.

14. Incidentes e vulnerabilidades

14.1 Resposta a incidentes

1. Detectar e classificar o evento; preservar evidências e registrar horário UTC.
2. Conter acesso, revogar credenciais e impedir propagação sem destruir evidências.
3. Avaliar tenants, jurisdições, dados, regras e decisões afetadas.
4. Notificar responsáveis conforme matriz legal, contratual e corporativa.
5. Erradicar causa, recuperar de forma verificada e monitorar recorrência.
6. Executar análise de causa, ações corretivas com prazos e atualização desta especificação quando necessário.

14.2 SLAs de vulnerabilidade propostos

Severidade	Tratamento inicial	Correção/mitigação-alvo
Crítica explorável	Imediato; avaliar suspensão/bloqueio	24 a 72 horas conforme exposição
Alta	Até 1 dia útil	Até 7 dias corridos
Média	Até 5 dias úteis	Até 30 dias
Baixa	Backlog priorizado	Até 90 dias ou justificativa

Os SLAs acima são proposta inicial e devem ser alinhados à política corporativa; risco aceito exige proprietário, compensação, expiração e aprovação.

15. Governança geral e responsabilidades

15.1 Artefatos obrigatórios

- SECURITY.md, CONTRIBUTING.md, CODEOWNERS e política de classificação de dados.
- ADRs para decisões arquiteturais, RACI, matriz de acesso e inventário de ativos/dados.
- Runbooks de incidente, restore, deploy, rollback/forward-fix e mudança emergencial.

- Registro de riscos, exceções, vulnerabilidades, dependências, SBOM e evidências de portões.
- Políticas de retenção/descarte, gestão de segredos, migrações e versionamento de regras.

15.2 RACI mínimo

Atividade	R	A	C	I
Controle de acesso	Engenharia/ Segurança	Segurança	Produto/Privacidade	Auditoria
Regra tributária	Management Tax	Aprovador tributário	Jurídico/Produto	Engenharia/ Auditoria
Migração de banco	Engenharia	Líder técnico	DB/Segurança/ Produto	Operações
Release produtivo	Engenharia/ Operações	Responsável por produção	Segurança/Produto/ Tax	Auditoria
Incidente	Incident Commander	Segurança/Negócio	Jurídico/ Privacidade/ Operações	Partes afetadas
Exceção de controle	Proponente	Dono do risco	Segurança/Jurídico/ Produto	Auditoria

16. Portões obrigatórios de entrega

Gate	Saída obrigatória	Evidência mínima	Aprovação
0 — Fundação	Classificação, ameaça e arquitetura de segurança aprovadas	Inventário, diagrama de confiança, threat model, riscos	Produto + Segurança
1 — Identidade	Autenticação, tenant e autorização funcionam com dados sintéticos	Matriz de acesso e testes positivos/negativos	Engenharia + Segurança
2 — Auditoria e isolamento	RLS, trilha imutável e exports testados	Testes tenant A/B, eventos e revisão adversarial	Segurança + Auditoria + Produto
3 — Migração e CI	Pipeline protegido e migração reproduzível	PR, banco descartável, checksum, scans e approvals	Engenharia + Plataforma
4 — Regras	Regras versionadas, aprovadas e reproduzíveis	Casos, replay histórico, checksum e segregação	Management Tax + Produto
5 — Resiliência	Backup, restore e incidente exercitados	Relatórios de teste, RPO/RTO e ações	Operações + Segurança
6 — Revisão independente	Pentest/revisão adversarial e privacidade concluídos	Relatório, correções e riscos residuais	Segurança + Privacidade
7 — Produção	Prontidão, operação e risco residual formalmente aceitos	Checklist completo, runbooks, owners, rollback	Comitê de produção

BLOQUEIO Dados reais só podem ingressar após o Gate 2. O uso produtivo somente pode ocorrer após o Gate 7. Aprovação verbal, urgência comercial ou existência de antivírus não substituem evidência de gate.

17. Catálogo de controles prioritários

ID	Requisito	Evidência	Owner	Gate
SEC-IAM-001	Negar por padrão e exigir identidade corporativa/MFA	Teste de acesso + configuração do IdP	Segurança	1
SEC-IAM-002	Separar contas humanas, técnicas e privilegiadas	Inventário, owners e revisão de acesso	Segurança	1
SEC-DAT-001	Aplicar tenant_id e RLS forçada no PostgreSQL	DDL/policies + testes A/B	Engenharia	2
SEC-DAT-002	Proibir dados reais em desenvolvimento	Pipeline/datasets + revisão	Produto/ Privacidade	2
SEC-AUD-001	Registrar eventos sensíveis append-only	Schema, eventos e retenção	Engenharia/ Auditoria	2
SEC-AUD-002	Detectar adulteração e correlacionar camadas	Hash/encadeamento e correlation_id	Segurança	2
SEC-SEC-001	Manter segredos em vault e rotacionáveis	Configuração, teste de rotação	Plataforma	3
SEC-PLT-001	Proteger GitHub, branch e pipeline	Ruleset, CODEOWNERS e checks	Engenharia	3
SEC-PLT-002	Executar containers não root e mínimos	Dockerfile, scan, teste runtime	Plataforma	3
GOV-MIG-001	Versionar e revisar toda migração Prisma	PR, SQL, checksums e banco descartável	Engenharia	3
GOV-MIG-002	Usar expand-contract e plano de forward-fix	Migration plan e teste de compatibilidade	Engenharia	3
GOV-RUL-001	Publicar regra imutável e versionada	Modelo, workflow, checksum	Management Tax	4
GOV-RUL-002	Reproduzir decisão histórica	Teste de replay com versão fixada	Produto/Tax	4
GOV-OPS-001	Testar backup e restauração	Relatório de restore	Operações	5
GOV-OPS-002	Manter runbook e simular incidente	Exercício e ações corretivas	Segurança/Ops	5
GOV-REL-001	Promover artefato imutável com SBOM	Digest, assinatura, SBOM e release	Plataforma	7

18. Estratégia de testes e Definition of Done

18.1 Famílias de testes

- Unitários: validação, cálculo, autorização contextual e estados de regra.
- Integração: PostgreSQL/RLS, Prisma, filas, storage, identidade e auditoria.
- Contrato: APIs, schemas, erros sem vazamento e compatibilidade.
- Migração: banco vazio, versão anterior, dados sintéticos em volume, locks e forward-fix.
- Segurança: SAST, SCA, segredo, imagem, DAST, abuso de autorização e threat model.
- Resiliência: backup/restore, perda de dependência, retry/idempotência e recuperação.
- Regras: casos normais, bordas, datas, retroatividade, regressão e replay histórico.

18.2 Definition of Done de qualquer feature

- Critérios funcionais e de segurança definidos e revisados.

- Autorização positiva e negativa testada; isolamento multitenant comprovado quando aplicável.
- Eventos de auditoria definidos, sem segredos e com correlation_id.
- Migração e compatibilidade avaliadas; dados e regras versionados.
- Testes automatizados aprovados e cobertura de risco suficiente.
- Documentação, runbook, métricas, alertas e owner atualizados.
- Nenhuma vulnerabilidade bloqueadora nem exceção expirada.

19. Registro inicial de riscos e decisões abertas

ID	Risco/decisão	Tratamento proposto	Owner	Prazo
R-001	Acesso cruzado entre tenants	RLS forçada, autorização contextual e testes adversariais	Engenharia/Segurança	Antes do Gate 2
R-002	Uso indevido do Prisma Studio	Bloqueio de rede/credencial e política local-only	Engenharia	Imediato
R-003	Migração destrutiva ou não reproduzível	Expand-contract, checksum, banco descartável e approval	Engenharia	Gate 3
R-004	Regra alterada sem rastreabilidade	Versão imutável, workflow e checksum	Management Tax	Gate 4
R-005	Segredo em Git/imagem/log	Vault, scanners, rotação e revisão	Plataforma	Gate 3
D-001	Definir provedor de identidade e MFA	ADR com integração e lifecycle	Produto/Segurança	Gate 0
D-002	Definir RPO/RTO e retenção por classe	Workshop com negócio, Jurídico e Operações	Produto	Gate 0
D-003	Definir hospedagem e cofre de segredos	ADR, modelo de custos e threat model	Plataforma	Gate 0

20. Plano de implantação 30/60/90 dias

0–30 dias — Fundação

- Aprovar classificação, threat model, arquitetura de referência, RACI e decisões abertas do Gate 0.
- Configurar repositório privado, ruleset, CODEOWNERS, scanners e padrão de ADR.
- Definir identidade, tenant context, modelo de autorização, schema de auditoria e políticas RLS.
- Criar baseline de WSL2/VS Code/Docker e proibir Prisma Studio fora do local sintético.

31–60 dias — Controles executáveis

- Implementar autenticação/autorização, isolamento, auditoria e testes tenant A/B.
- Implantar pipeline de migração, banco descartável, scans, SBOM e artefatos imutáveis.
- Implementar modelo e workflow de regras versionadas com casos de teste e replay.
- Documentar runbooks, política de segredo, backup/restore e vulnerabilidades.

61–90 dias — Prova de prontidão

- Executar testes de restauração, incidente, migração de volume e mudança emergencial.

- Realizar revisão adversarial independente, pentest proporcional e avaliação de privacidade.
- Corrigir achados, registrar riscos residuais e completar evidências dos Gates 0 a 6.
- Submeter prontidão produtiva ao Gate 7; somente depois autorizar dados e operação conforme política.

Apêndice A — Schema de referência do evento de auditoria

Campo	Tipo	Obrigatório	Observação
event_id	UUID	Sim	Identificador global imutável
event_type	string versionada	Sim	Tipo estável do evento
occurred_at_utc	timestamp UTC	Sim	Momento do fato
actor_id / actor_type	string	Sim	Humano, serviço ou processo
tenant_id	UUID	Condicional	Escopo do tenant
jurisdiction_id	UUID	Condicional	Escopo regulatório
resource_type / resource_id	string	Sim	Objeto afetado
action / result	enum	Sim	Operação e resultado
reason_code / reason_text	string	Condicional	Justificativa controlada
before_ref / after_ref	JSON/ref	Condicional	Diferença segura, sem segredo
correlation_id / session_id	string	Sim	Correlação ponta a ponta
rule_id / rule_version	string	Condicional	Regra efetivamente utilizada
source_ip / client	string	Condicional	Origem conforme privacidade
integrity_hash / prev_hash	hash	Sim	Deteção de adulteração

Apêndice B — Checklist de migração

- [] Impacto funcional, regulatório, RLS, auditoria, volume e lock documentado.
- [] SQL gerado revisado; nenhuma migração aplicada foi alterada.
- [] Banco descartável criado desde zero e atualizado a partir da versão anterior.
- [] Compatibilidade expand-contract e rollout/feature flag avaliados.
- [] Backfill idempotente, observável e em lotes quando necessário.
- [] Backup, restore e forward-fix definidos e testados proporcionalmente.
- [] Aprovações de risco e janela de mudança registradas.
- [] prisma migrate deploy executado somente pelo pipeline controlado.
- [] Pós-deploy validou saúde, integridade, RLS, auditoria e métricas.

Apêndice C — Checklist de release e produção

- [] Commit, tag, imagem/digest e SBOM identificados.
- [] Testes funcionais, segurança, isolamento, migração e regras aprovados.
- [] Vulnerabilidades bloqueadoras ausentes; exceções vigentes e aprovadas.

- [] Segredos, chaves e acessos de produção preparados com menor privilégio.
- [] Migrações e regras compatíveis com a versão do serviço.
- [] Métricas, alertas, dashboards e correlation_id verificados.
- [] Backup/restore, runbook de incidente e plano de forward-fix disponíveis.
- [] Aprovação de environment e segregação de funções confirmadas.
- [] Gates requeridos possuem evidências e assinaturas válidas.
- [] Comunicação, janela e responsáveis de plantão confirmados.

Apêndice D — Critérios de revisão do Prisma Studio

Pergunta	Resposta exigida
O Studio possui credencial produtiva?	Não
A rede permite conexão produtiva a partir da estação?	Não
Os dados locais são reais?	Não; sintéticos ou anonimizados aprovados
Mudanças de schema são feitas pelo Studio?	Não; exclusivamente Prisma Migrate
Correções produtivas usam o Studio?	Não; aplicação auditada ou script governado
O uso local é documentado e revisável?	Sim; finalidade, owner e configuração

Apêndice E — Glossário

Termo	Definição
ABAC	Autorização baseada em atributos de identidade, dado e contexto.
ADR	Registro versionado de decisão arquitetural e suas consequências.
RLS	Row-Level Security do PostgreSQL para restringir linhas pelo contexto.
RPO / RTO	Perda de dados tolerada / tempo-alvo para recuperação.
SBOM	Inventário dos componentes e dependências do artefato de software.
Tenant	Unidade organizacional isolada logicamente no sistema.
Forward-fix	Correção posterior controlada que preserva compatibilidade e histórico.
Legal hold	Suspensão de descarte por obrigação legal, investigação ou auditoria.